Sign Language MNIST Classification Report

From a Fully Connected Baseline to an Improved Convolutional Neural Network

Applied Neural Networks Project Report

Group member: Jia Liu, Linyi Xia, Yuwen Cen

1. Project Objective and Data Overview

The objective of this project is to classify hand gestures from the Sign Language MNIST dataset into corresponding alphabet letters and to evaluate how model design influences predictive performance.

This task is inherently challenging because many hand gestures are visually similar, differing only in subtle finger positions or orientations. Therefore, successful classification requires models that can capture fine-grained spatial patterns in image data.

In this project, we first establish a baseline using a Multi-Layer Perceptron (MLP), which treats each image as a flattened vector and does not explicitly account for spatial structure. We then introduce Convolutional Neural Networks (CNNs), which are designed to exploit spatial relationships in image data. By comparing these models, we aim to understand how incorporating spatial information impacts classification accuracy and generalization performance.

The dataset contains 27,455 training examples and 7,172 test examples. Each observation is a 28×28 grayscale image flattened into 784-pixel features, plus one class label. The labels are fairly balanced across classes shown in Figure 1, which reduces the risk that overall accuracy is driven by a small number of dominant categories. Note that the label is from 0 to 24, excluding 9(J) and 25(Z) because the corresponding sign languages are dynamic and thus cannot be captured by graph only.

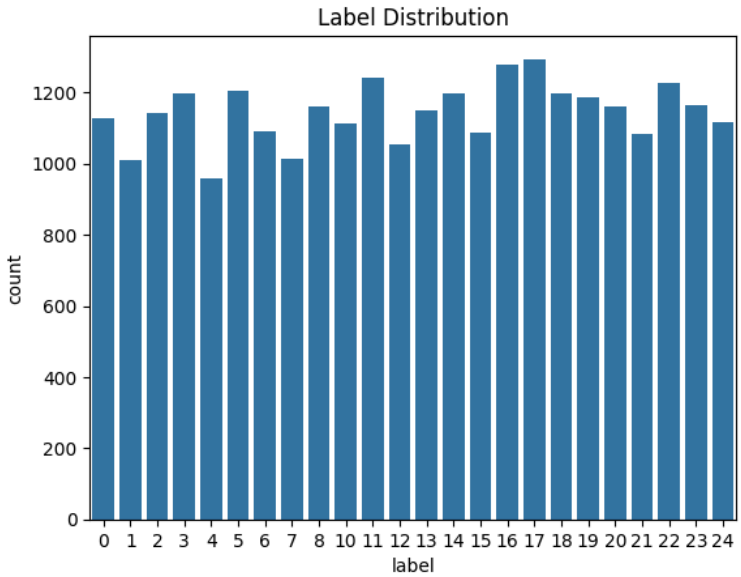


Figure 1. Class frequencies in the training set are reasonably balanced.

To better understand the dataset, we visualize several sample images from the training set. Each image corresponds to a hand gesture representing a sign language letter shown in Figure 2.

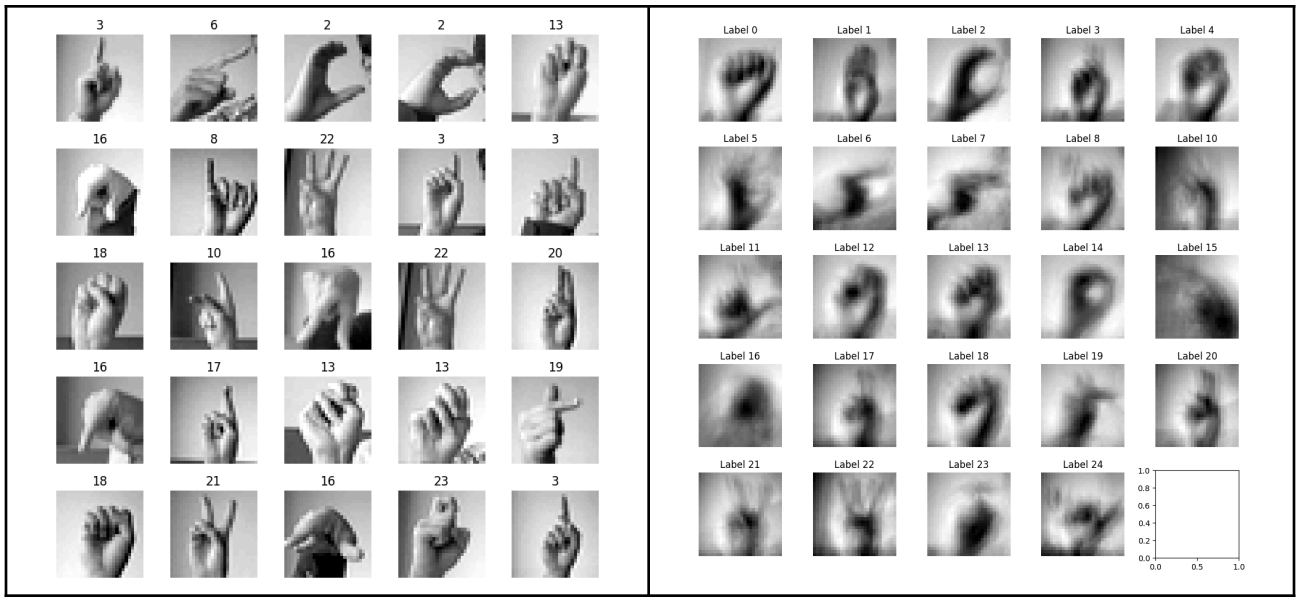


Figure 2. Example hand-sign images (left) and class-average images (right). Several classes have similar visual structure, foreshadowing later confusion errors.

2. Data Preprocessing

To ensure reproducibility, we set the random seed to be 42. Pixel intensities were scaled from 0–255 to 0–1, which stabilizes optimization and makes gradients more well-behaved. The dataset already came with an official training/test split. To tune model hyperparameters, we further split the training data into training and validation subsets using a stratified 85/15 split. This preserved class balance while leaving the official test set untouched for final evaluation.

Two feature representations were used. For the MLP baseline, each image remained as a flattened 784-dimensional vector. For the CNN models, the same images were reshaped to 28×28×1 so that convolutional filters could exploit local spatial structure. Since the class labels were not consecutive integers in the raw file, they were remapped to values from 0 to 23 before training.

3. Baseline Model: Multi-Layer Perceptron (MLP)

The first model was a multi-layer perceptron trained on flattened pixel vectors. This baseline serves two purposes. First, it establishes whether a straightforward feedforward network can learn meaningful signals from the raw pixels. Second, it provides a reference point for judging whether convolution really adds value.

The MLP model takes flattened 28×28 images as input, resulting in 784 input features. The network consists of two hidden layers with 256 and 128 units, both using ReLU activation functions, with dropout applied after each hidden layer to reduce overfitting. The output layer contains 24 units corresponding to the 24 gesture classes.

We initially considered simpler architectures, but increasing the depth to two hidden layers improved both training stability and classification performance, so this configuration was selected as the final baseline model.

The baseline MLP model learned quickly and achieved approximately 92% training accuracy and a best validation accuracy of 98.83%. The steadily increasing accuracy along with decreasing loss curves indicate that the model successfully learned useful patterns from the training data.

Model	Best Validation Accuracy	Test Accuracy
MLP	0.9883	0.7765

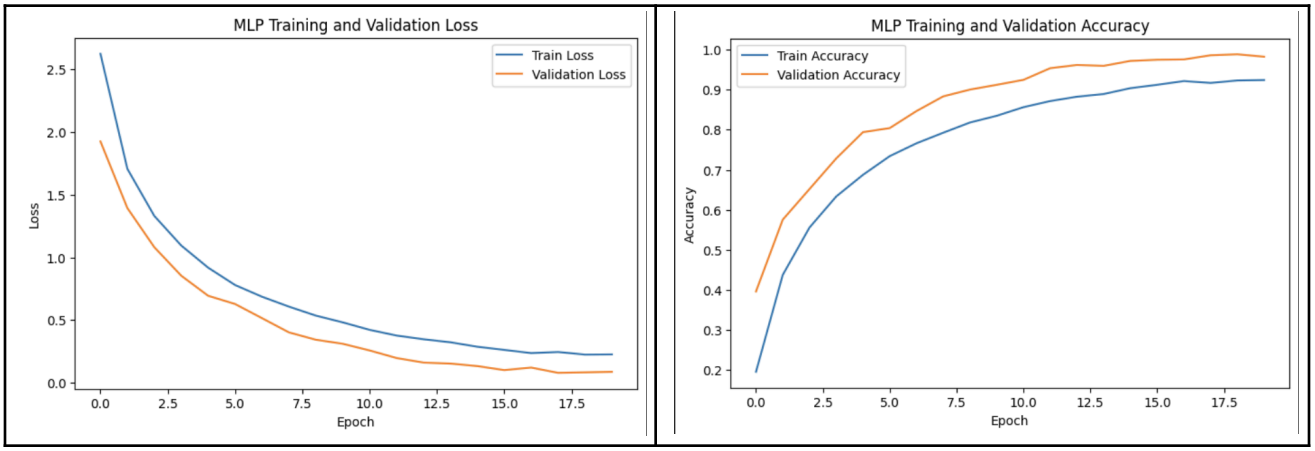


Figure 3. MLP training and validation curves.

The training curves show that both training and validation accuracy increase steadily, while loss decreases smoothly, indicating stable learning. The validation accuracy remains consistently higher than the training accuracy, suggesting that the model fits the training-like data well.

However, the test accuracy is only about 78%, which is significantly lower than the validation accuracy. This difference suggests that the model does not generalize well to the official test set.

One possible explanation is that the validation set was created by splitting the training data and therefore shares a similar distribution. In contrast, the official test set may contain images with slightly different characteristics, making the classification task more challenging.

Confusion Matrix Analysis

To better understand the classification performance of the baseline model, we visualize a confusion matrix on the test set.

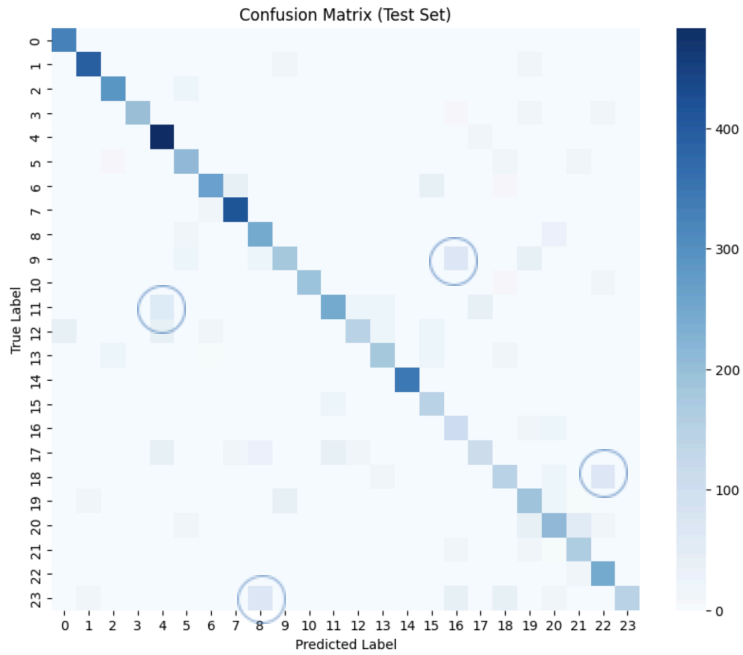


Figure 4. MLP confusion matrix on the official test set. Errors are concentrated in visually similar classes rather than being uniformly random.

The confusion matrix above shows that most predictions fall along the diagonal, indicating that the model correctly classifies many samples. However, several off-diagonal entries reveal that certain hand signs are frequently confused with visually similar gestures. Some confused pairs are: 23(True)→8(Predict) (67 samples), 9→16 (65), 18→22 (64), 11→ 4(62), circled in Figure 4, which are classes whose hand silhouettes differ only in local finger position .

This suggests that the baseline MLP model struggles to capture subtle spatial patterns in the images. Since MLP treats the image as a flattened vector, it cannot effectively exploit the spatial structure of hand shapes. This limitation motivates the use of convolutional neural networks (CNNs) in the next stage of the project.

4. CNN Model Development

CNNs are better suited for image tasks because they detect local patterns such as edges, shapes, and spatial motifs before combining them into higher-level features. In this section, the CNN stage was developed in two rounds: a baseline CNN with two convolutional blocks and an improved CNN with additional depth.

4.1 Baseline CNN

The baseline CNN used two convolutional layers, ReLU activations, max pooling, and dropout before the final classifier. This model already represented a substantial conceptual improvement over the MLP because feature extraction was no longer left entirely to the dense layers. Instead, the network could learn small local filters that slide across the image and preserve spatial information.

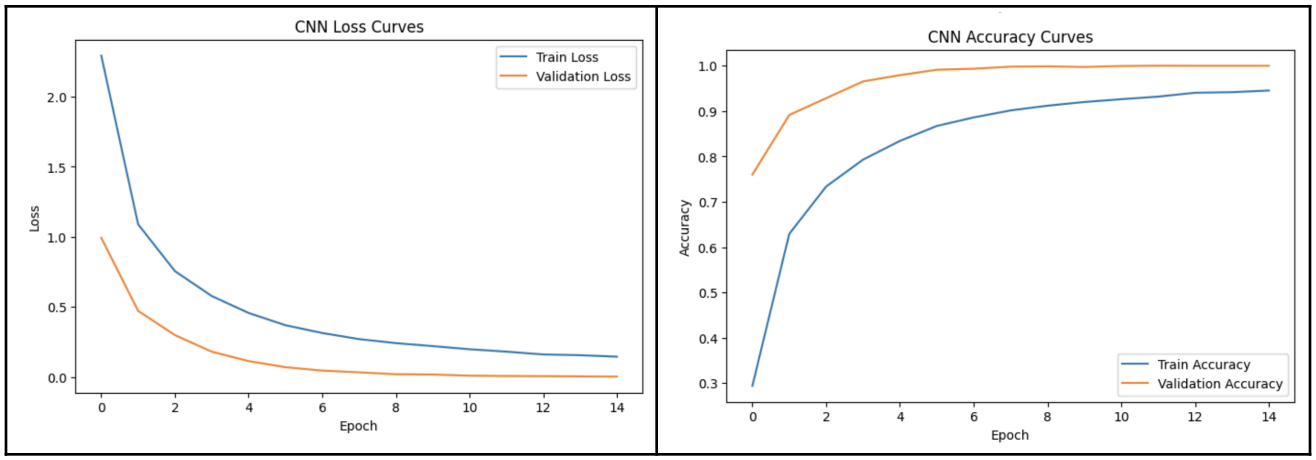


Figure 5. Baseline CNN loss and accuracy curves. Compared with the MLP, the network learns more smoothly and reaches much stronger test performance.

From the training curves, both training and validation loss decrease smoothly, indicating stable optimization. At the same time, validation accuracy increases rapidly and remains close to training accuracy, suggesting that the model learns effectively without severe overfitting.

The baseline CNN performed well with training and validation loss both decreased steadily over 15 epochs, and training accuracy rose to 94.52% and validation accuracy reached 99.98%. The final test accuracy was 91.30%, a jump of more than 16 percentage points over the MLP. This shows that the CNN can effectively capture spatial features in the hand gesture images and is better suited for image classification compared to MLP. However, since the test accuracy is still lower than the validation accuracy, there is room for improvement in generalization.

4.2 Improved CNN

To improve the baseline CNN, we increased the depth of the convolutional feature extractor by adding an additional convolutional layer. This allows the network to learn more complex spatial features from the hand gesture images while keeping the training settings consistent with the baseline model. All other hyperparameters were kept unchanged.

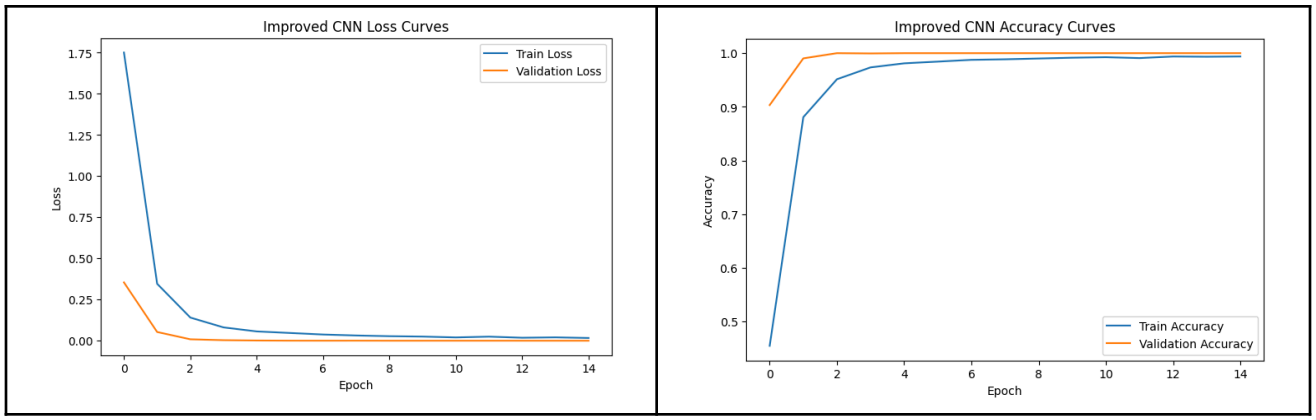


Figure 6. Improved CNN training curves.

The improved CNN shows stable training behavior, with both loss decreasing and accuracy increasing smoothly. The gap between training and validation performance remains small, indicating good generalization.

The improved CNN increased test accuracy to 94.69%, compared to 91.30% for baseline CNN, indicating better generalization on the unseen test data. The additional convolutional depth allows the network to learn richer spatial features from the hand gesture images, leading to improved classification performance. Although the absolute gain over the baseline CNN is smaller than the jump from MLP to CNN, suggesting diminishing returns as model complexity increases, it is still meaningful because it comes on top of an already strong model.

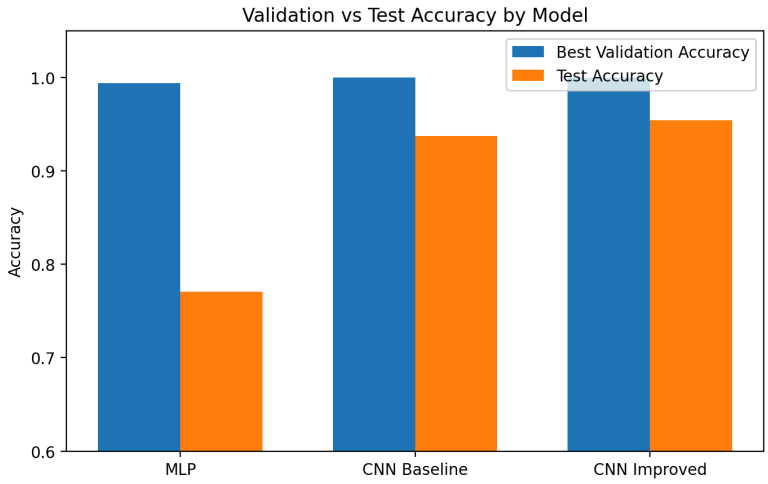


Figure 7. Best validation versus test accuracy across the three models.

Confusion Matrix Analysis

To further evaluate the CNN model, we also examine its confusion matrix on the test set. Compared to the MLP, the CNN shows fewer off-diagonal errors, indicating improved classification performance.

In particular, several previously confused class pairs are better separated, suggesting that the CNN is more effective at capturing subtle spatial differences between similar hand gestures.

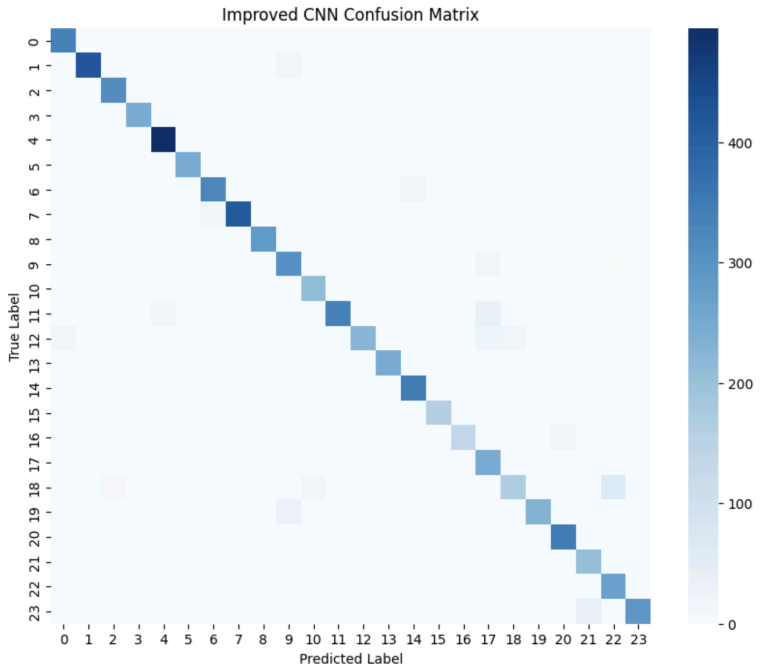


Figure 8. Improved CNN Confusion Matrix

5. Finding & Conclusion

A key finding of this project is the large gap between validation and test performance. The MLP achieved very high validation accuracy, but its test accuracy dropped to around 78%, showing poor generalization. This gap suggests that the validation set, which was split from the training data, has a very similar distribution and may give overly optimistic results. In contrast, the official test set appears to be slightly more challenging, revealing the true performance of the model. This also suggests that a random train-validation split may not be sufficient for evaluating generalization in image classification tasks.

From the confusion matrix, we also observe that most errors occur between visually similar classes. This indicates that the MLP struggles to capture small local differences in hand shapes. Since the MLP treats images as flattened vectors, it ignores spatial relationships, which are important for this task. After switching to CNN, the model performance improved significantly. This shows that capturing spatial structure is essential for image classification problems like this one.

In conclusion, this project shows that model design plays a key role in image classification tasks. While the MLP provides a useful baseline, it cannot effectively capture spatial patterns, leading to poor generalization. The CNN models greatly improve performance by learning spatial features from the images. This leads to a large increase in test accuracy, from 77.65% to 91.3%, and further improvement to 94.69% with a deeper model. This project also highlights that validation accuracy alone can be misleading. A model that performs well on validation data may still fail on unseen test data if the distributions are different. Overall, the results show a clear improvement from MLP to CNN, demonstrating the importance of using structure-aware models for image data.

Code Report

1. Import Libraries

In [ ]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import torch
import torch.nn as nn
import random
from torch.utils.data import TensorDataset, DataLoader
from sklearn.model_selection import train_test_split

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

set_seed(42)
```

2. Load Dataset

In [ ]:
```
train_df = pd.read_csv("/sign_mnist_train.csv")
test_df = pd.read_csv("/sign_mnist_test.csv")
```

3. EDA

In [ ]:
```
print(train_df.shape)
print(test_df.shape)

train_df.head()
```

```
(27455, 785)
(7172, 785)
```

Out[ ]:

	label	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	pixel9	...	pixel775	pixel776	pixel777	pixel778
0	3	107	118	127	134	139	143	146	150	153	...	207	207	207	207
1	6	155	157	156	156	156	157	156	158	158	...	69	149	128	87
2	2	187	188	188	187	187	186	187	188	187	...	202	201	200	199
3	2	211	211	212	212	211	210	211	210	210	...	235	234	233	231
4	13	164	167	170	172	176	179	180	184	185	...	92	105	105	108

5 rows × 785 columns

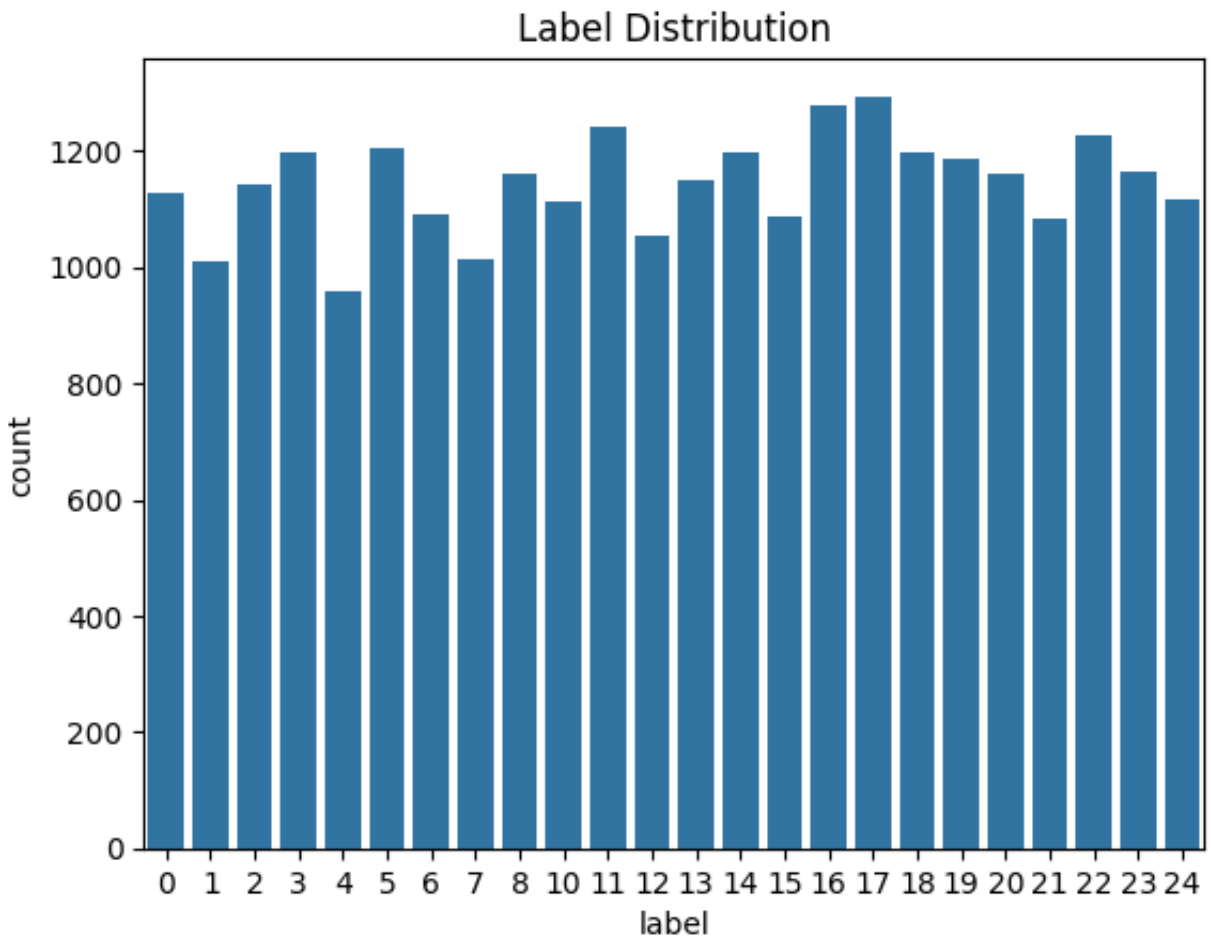
The dataset contains 27,455 training samples and 7,172 test samples. Each image is represented by 784 pixel features (28×28 grayscale image) and one label column.

In [ ]:
```
train_df.isnull().sum().sum()
test_df.isnull().sum().sum()
```

Out[ ]:
```
np.int64(0)
```

No missing values were found in the dataset.

In [ ]:
```
sns.countplot(x="label", data=train_df)
plt.title("Label Distribution")
plt.show()
```



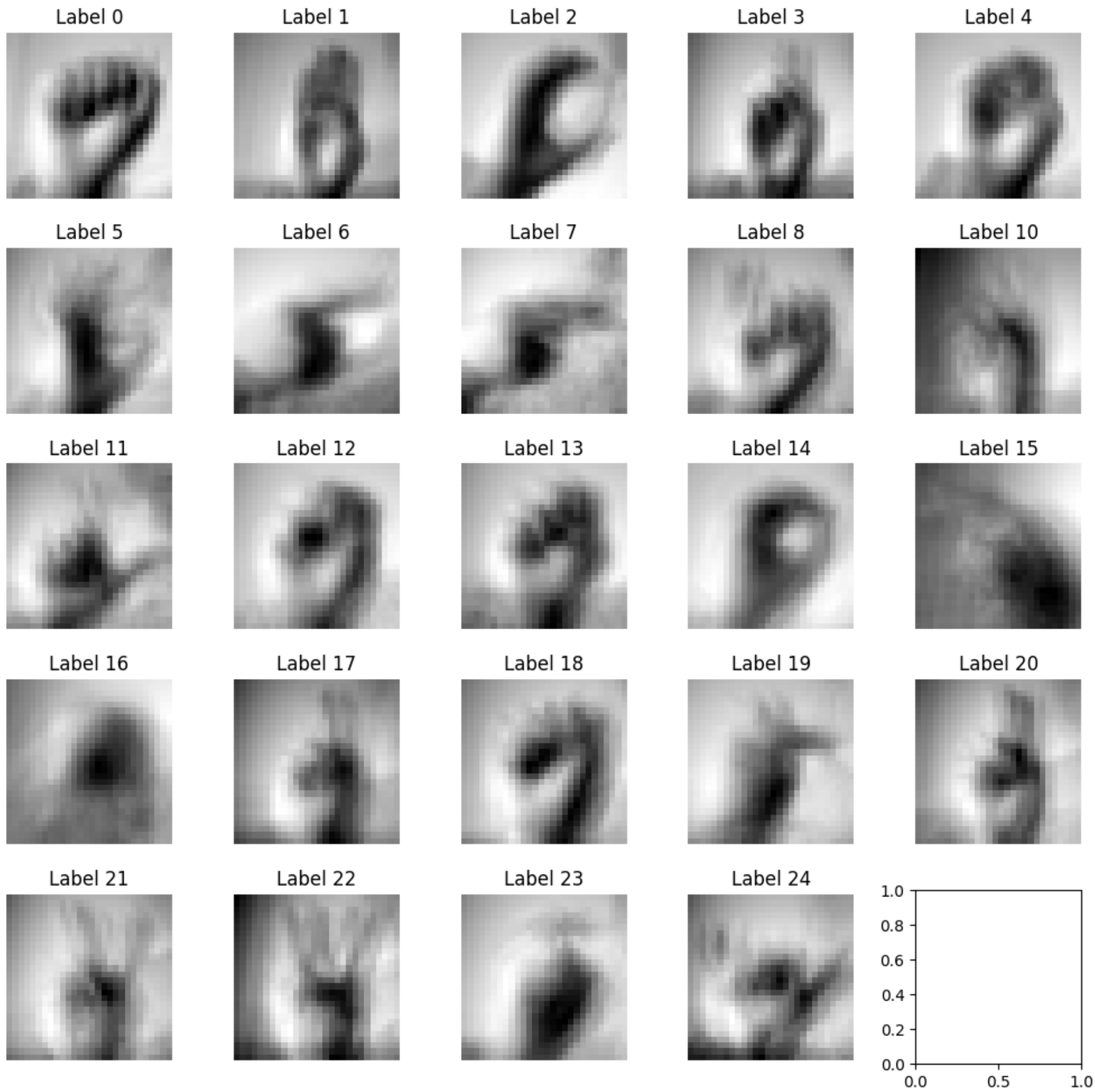
The label distribution appears relatively balanced across classes, which is beneficial for model training.

To better understand the dataset, we visualize several sample images from the training set. Each image corresponds to a hand gesture representing a sign language letter.

In [ ]:
```
fig, axes = plt.subplots(5,5, figsize=(8,8))

for i, ax in enumerate(axes.flat):
    image = train_df.iloc[i,1:].values.reshape(28,28)
    label = train_df.iloc[i,0]

    ax.imshow(image, cmap="gray")
    ax.set_title(label)
    ax.axis("off")

plt.tight_layout()
plt.show()
```

To explore the general pattern of each class, we compute the average image for every label by averaging the pixel values across all images belonging to the same class.

In [ ]:
```
fig, axes = plt.subplots(5,5, figsize=(10,10))

labels = sorted(train_df["label"].unique())

for i, label in enumerate(labels):

    class_images = train_df([train_df["label"] == label]
                   .iloc[:,1:]
                   .values
    )
    avg_image = class_images.mean(axis=0).reshape(28,28)

    ax = axes[i//5, i%5]
    ax.imshow(avg_image, cmap="gray")
    ax.set_title(f"Label {label}")
    ax.axis("off")

plt.tight_layout()
plt.show()
```



The average images reveal consistent hand shapes within each class. Some classes appear visually similar, which may lead to classification challenges for the model.

4. Data Preprocessing

Pixel Normalization:

The pixel values in the images range from 0 to 255. To stabilize neural network training and improve optimization performance, the pixel values were normalized to the range [0,1] by dividing all pixel intensities by 255.

In [ ]:
```
X_train = train_df.drop("label", axis=1) / 255.0
y_train = train_df["label"]

X_test = test_df.drop("label", axis=1) / 255.0
y_test = test_df["label"]
```

In [ ]:
```
print(X_train.min().min(), X_train.max().max())
```

```
0.0 1.0
```

The dataset already provides predefined training and testing sets. To tune model hyperparameters, we further split the training data into training and validation subsets.

In [ ]:
```
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X_train,
```

```
    y_train,
    test_size=0.15,
    random_state=42,
    stratify=y_train
)
```

In [ ]:
```
print(X_train.shape)
print(X_val.shape)
print(X_test.shape)
```

```
(23336, 784)
(4119, 784)
(7172, 784)
```

Image Reshaping for CNN Input

Since each image consists of 784 pixels representing a 28×28 grayscale image, the feature vectors were reshaped into 28×28 matrices with one channel to be compatible with convolutional neural networks.

In [ ]:
```
X_train_img = X_train.values.reshape(-1, 28, 28, 1)
X_val_img = X_val.values.reshape(-1, 28, 28, 1)
X_test_img = X_test.values.reshape(-1, 28, 28, 1)

print(X_train_img.shape)
print(X_val_img.shape)
print(X_test_img.shape)
```

```
(23336, 28, 28, 1)
(4119, 28, 28, 1)
(7172, 28, 28, 1)
```

5. Baseline Model: Multi-Layer Perceptron (MLP)

As a baseline model, we implemented a simple multi-layer perceptron (MLP) using flattened 784-dimensional pixel vectors. Since the original class labels are not consecutively numbered, they were remapped to integers from 0 to 23 before training.

In [ ]:
```
# Remap labels to 0, 1, ..., 23
unique_labels = sorted(train_df["label"].unique())
label_map = {
    old_label: new_label
    for new_label, old_label in enumerate(unique_labels)
}

y_train_mapped = y_train.map(label_map)
y_val_mapped = y_val.map(label_map)
y_test_mapped = y_test.map(label_map)

print("Original labels:", unique_labels)
print("Mapped labels:", sorted(y_train_mapped.unique()))
print("Number of classes:", len(unique_labels))

# Convert to tensors
X_train_tensor = torch.tensor(X_train.values, dtype=torch.float32)
y_train_tensor = torch.tensor(y_train_mapped.values, dtype=torch.long)

X_val_tensor = torch.tensor(X_val.values, dtype=torch.float32)
y_val_tensor = torch.tensor(y_val_mapped.values, dtype=torch.long)

X_test_tensor = torch.tensor(X_test.values, dtype=torch.float32)
y_test_tensor = torch.tensor(y_test_mapped.values, dtype=torch.long)

# Datasets and loaders
train_dataset = TensorDataset(X_train_tensor, y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, y_test_tensor)

train_loader = DataLoader(train_dataset, batch_size=128, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=128, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=128, shuffle=False)

# Define MLP
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.model = nn.Sequential(
            nn.Linear(784, 256),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(256, 128),
```

```python
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(128, 24)
        )

    def forward(self, x):
        return self.model(x)

model = MLP()


# Loss and optimizer
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Training loop
num_epochs = 20

train_losses = []
val_losses = []
train_accuracies = []
val_accuracies = []

best_val_acc = 0.0
best_model_state = None

for epoch in range(num_epochs):
    # Train
    model.train()
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()

        outputs = model(X_batch)
        loss = loss_fn(outputs, y_batch)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs, 1)
        total_train += y_batch.size(0)
        correct_train += (predicted == y_batch).sum().item()

    epoch_train_loss = running_loss / len(train_loader)
    epoch_train_acc = correct_train / total_train

    # Validation
    model.eval()
    running_val_loss = 0.0
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            outputs = model(X_batch)
            loss = loss_fn(outputs, y_batch)

            running_val_loss += loss.item()

            _, predicted = torch.max(outputs, 1)
            total_val += y_batch.size(0)
            correct_val += (predicted == y_batch).sum().item()

    epoch_val_loss = running_val_loss / len(val_loader)
    epoch_val_acc = correct_val / total_val

    train_losses.append(epoch_train_loss)
    val_losses.append(epoch_val_loss)
    train_accuracies.append(epoch_train_acc)
    val_accuracies.append(epoch_val_acc)

    # save best model
    if epoch_val_acc > best_val_acc:
        best_val_acc = epoch_val_acc
        best_model_state = model.state_dict()

    print(
```

```
        f"Epoch [{epoch+1}/{num_epochs}] | "
        f"Train Loss: {epoch_train_loss:.4f} | Train Acc: {epoch_train_acc:.4f} | "
        f"Val Loss: {epoch_val_loss:.4f} | Val Acc: {epoch_val_acc:.4f}"
    )

# load best model
model.load_state_dict(best_model_state)
```

```
Original labels: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(
6), np.int64(7), np.int64(8), np.int64(10), np.int64(11), np.int64(12), np.int64(13), np.int64(14), np.int6
4(15), np.int64(16), np.int64(17), np.int64(18), np.int64(19), np.int64(20), np.int64(21), np.int64(22), n
p.int64(23), np.int64(24)]
Mapped labels: [np.int64(0), np.int64(1), np.int64(2), np.int64(3), np.int64(4), np.int64(5), np.int64(6),
np.int64(7), np.int64(8), np.int64(9), np.int64(10), np.int64(11), np.int64(12), np.int64(13), np.int64(1
4), np.int64(15), np.int64(16), np.int64(17), np.int64(18), np.int64(19), np.int64(20), np.int64(21), np.in
t64(22), np.int64(23)]
Number of classes: 24
Epoch [1/20] | Train Loss: 2.6245 | Train Acc: 0.1954 | Val Loss: 1.9255 | Val Acc: 0.3962
Epoch [2/20] | Train Loss: 1.7069 | Train Acc: 0.4374 | Val Loss: 1.3938 | Val Acc: 0.5754
Epoch [3/20] | Train Loss: 1.3313 | Train Acc: 0.5557 | Val Loss: 1.0818 | Val Acc: 0.6519
Epoch [4/20] | Train Loss: 1.0952 | Train Acc: 0.6333 | Val Loss: 0.8545 | Val Acc: 0.7283
Epoch [5/20] | Train Loss: 0.9185 | Train Acc: 0.6874 | Val Loss: 0.6942 | Val Acc: 0.7939
Epoch [6/20] | Train Loss: 0.7799 | Train Acc: 0.7340 | Val Loss: 0.6285 | Val Acc: 0.8041
Epoch [7/20] | Train Loss: 0.6866 | Train Acc: 0.7661 | Val Loss: 0.5156 | Val Acc: 0.8466
Epoch [8/20] | Train Loss: 0.6077 | Train Acc: 0.7925 | Val Loss: 0.4021 | Val Acc: 0.8832
Epoch [9/20] | Train Loss: 0.5359 | Train Acc: 0.8180 | Val Loss: 0.3434 | Val Acc: 0.9002
Epoch [10/20] | Train Loss: 0.4819 | Train Acc: 0.8348 | Val Loss: 0.3111 | Val Acc: 0.9121
Epoch [11/20] | Train Loss: 0.4222 | Train Acc: 0.8563 | Val Loss: 0.2578 | Val Acc: 0.9245
Epoch [12/20] | Train Loss: 0.3768 | Train Acc: 0.8715 | Val Loss: 0.1973 | Val Acc: 0.9536
Epoch [13/20] | Train Loss: 0.3472 | Train Acc: 0.8824 | Val Loss: 0.1611 | Val Acc: 0.9616
Epoch [14/20] | Train Loss: 0.3230 | Train Acc: 0.8891 | Val Loss: 0.1531 | Val Acc: 0.9595
Epoch [15/20] | Train Loss: 0.2875 | Train Acc: 0.9038 | Val Loss: 0.1332 | Val Acc: 0.9718
Epoch [16/20] | Train Loss: 0.2623 | Train Acc: 0.9122 | Val Loss: 0.1010 | Val Acc: 0.9748
Epoch [17/20] | Train Loss: 0.2368 | Train Acc: 0.9215 | Val Loss: 0.1213 | Val Acc: 0.9755
Epoch [18/20] | Train Loss: 0.2456 | Train Acc: 0.9169 | Val Loss: 0.0792 | Val Acc: 0.9859
Epoch [19/20] | Train Loss: 0.2248 | Train Acc: 0.9230 | Val Loss: 0.0831 | Val Acc: 0.9883
Epoch [20/20] | Train Loss: 0.2265 | Train Acc: 0.9241 | Val Loss: 0.0874 | Val Acc: 0.9823
```

Out[ ]: <All keys matched successfully>

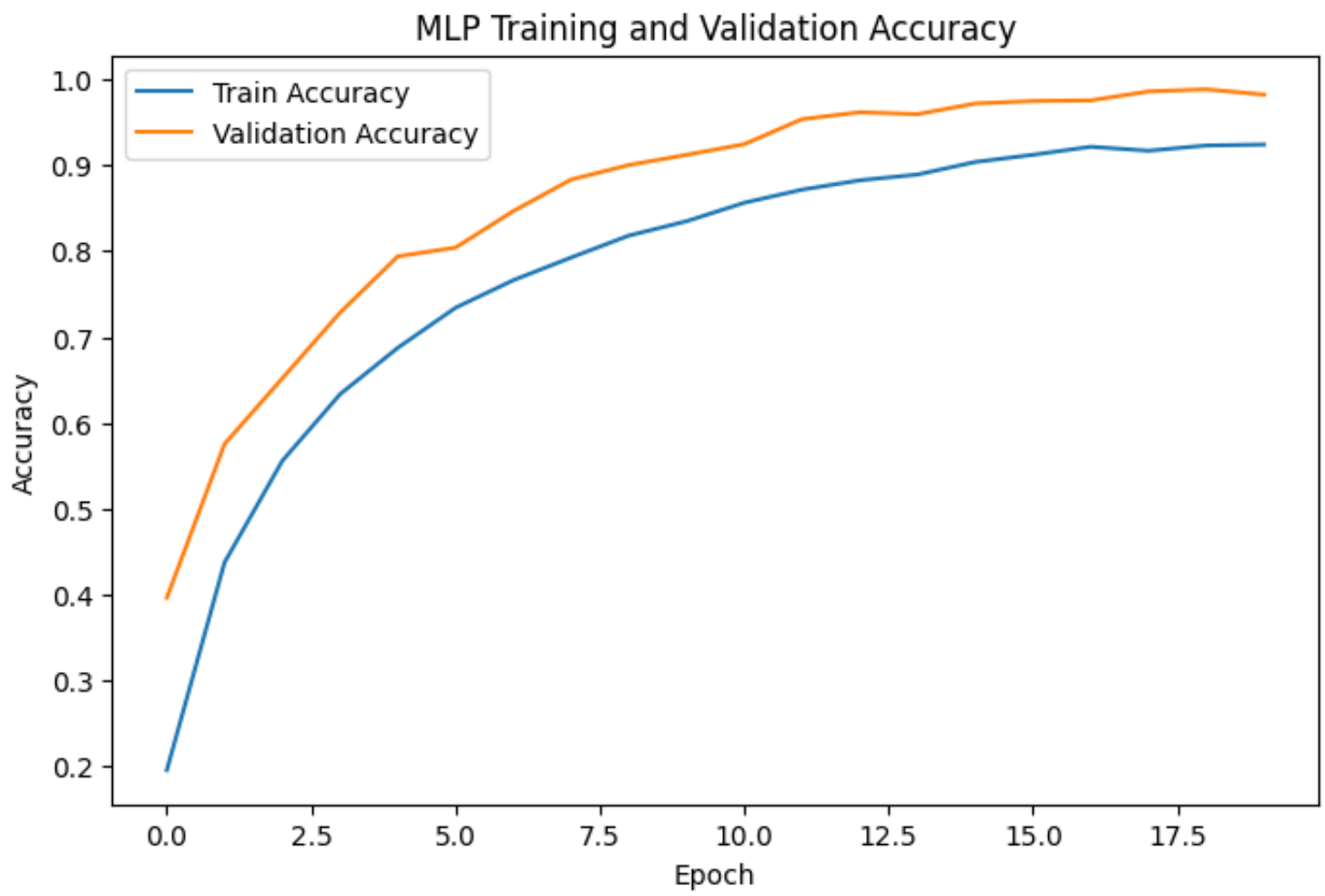
In [ ]:

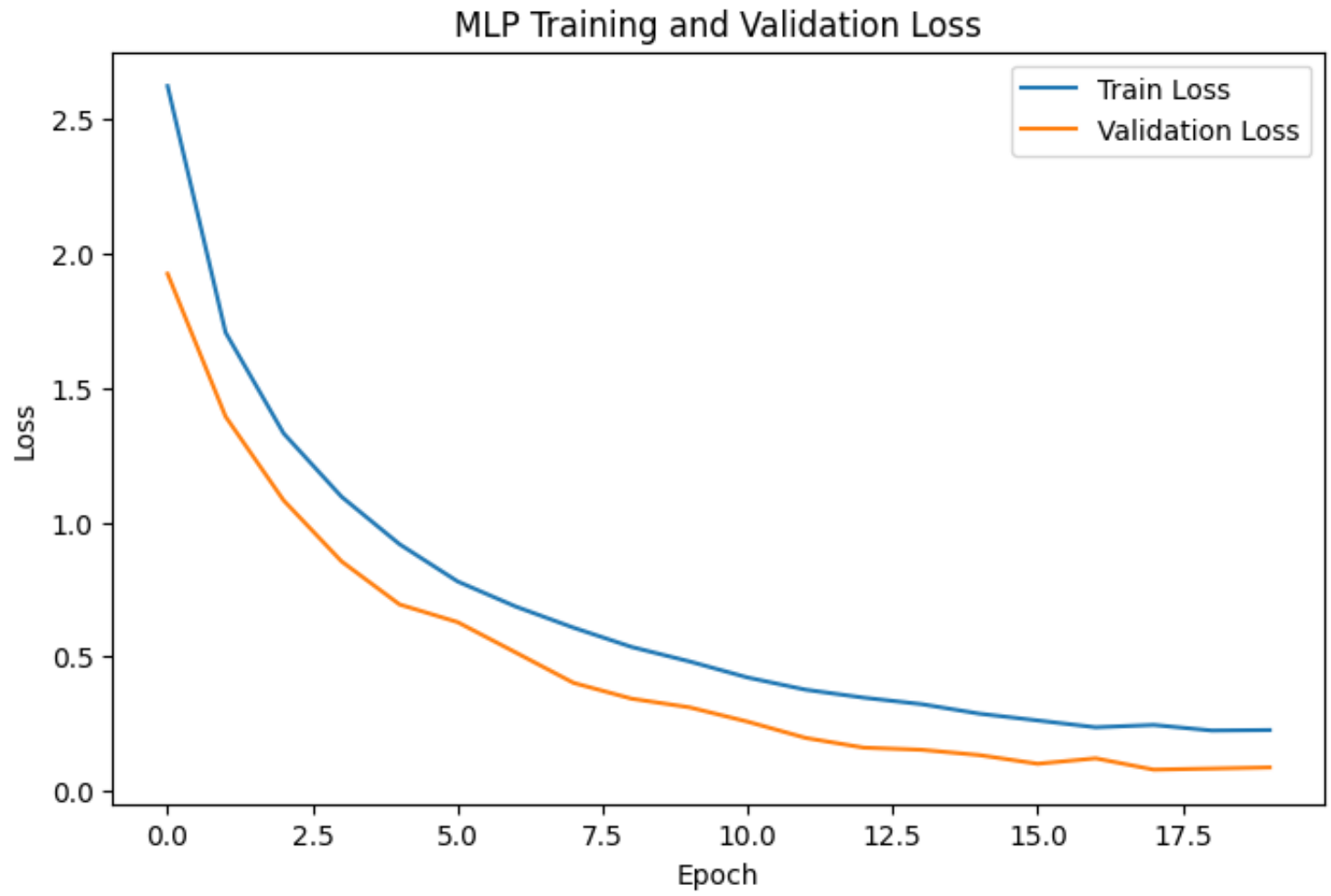
```
# Test evaluation
model.eval()
correct_test = 0
total_test = 0

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        outputs = model(X_batch)
        _, predicted = torch.max(outputs, 1)
        total_test += y_batch.size(0)
        correct_test += (predicted == y_batch).sum().item()

test_accuracy = correct_test / total_test
print(f"\nBest Validation Accuracy: {best_val_acc:.4f}")
print(f"Test Accuracy: {test_accuracy:.4f}")
```

```
Best Validation Accuracy: 0.9883
Test Accuracy: 0.7765
```

In [ ]:

```
# Plot accuracy curves
plt.figure(figsize=(8, 5))
plt.plot(train_accuracies, label="Train Accuracy")
plt.plot(val_accuracies, label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("MLP Training and Validation Accuracy")
plt.legend()
plt.show()

# Plot loss curves
plt.figure(figsize=(8, 5))
plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("MLP Training and Validation Loss")
plt.legend()
plt.show()
```





Baseline Model Result

The baseline MLP model achieved approximately 92.3% training accuracy and 98.8% validation accuracy. The steadily increasing accuracy and decreasing loss curves indicate that the model successfully learned useful patterns from the training data.

However, the test accuracy is only about 77.6%, which is significantly lower than the validation accuracy. This difference suggests that the model does not generalize well to the official test set.

One possible explanation is that the validation set was created by splitting the training data and therefore shares a similar distribution. In contrast, the official test set may contain images with slightly different characteristics, making the classification task more challenging.

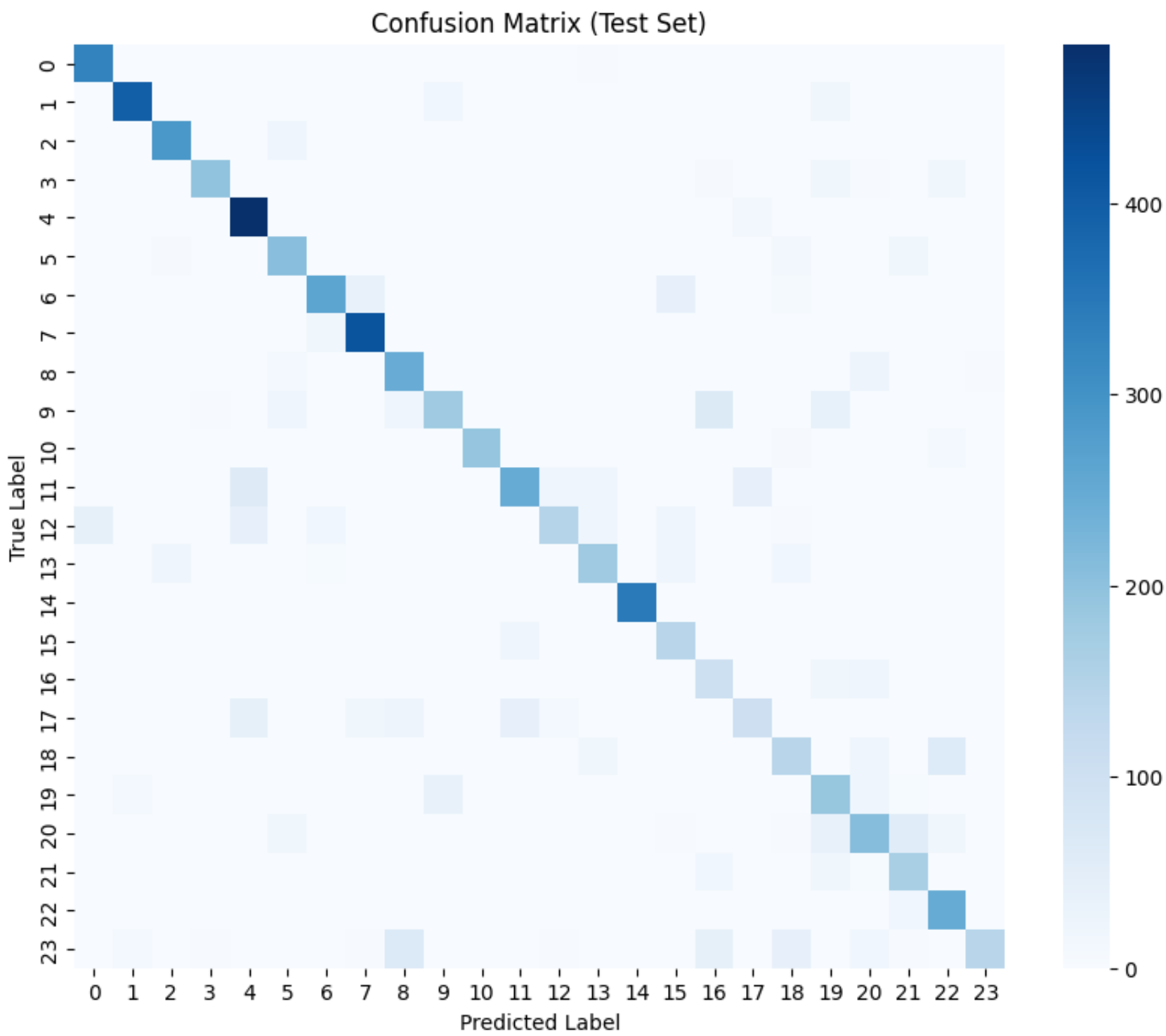
This result highlights the limitations of fully connected networks for image classification tasks and motivates the use of convolutional neural networks (CNNs) in the next stage.

Confusion Matrix Analysis

To better understand the classification performance of the baseline model, we visualize a confusion matrix on the test set.

In [ ]:
```
from sklearn.metrics import confusion_matrix
```

```
model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for X_batch, y_batch in test_loader:
        outputs = model(X_batch)
        _, predicted = torch.max(outputs, 1)

        all_preds.extend(predicted.numpy())
        all_labels.extend(y_batch.numpy())

cm = confusion_matrix(all_labels, all_preds)

plt.figure(figsize=(10,8))

sns.heatmap(
    cm,
    cmap="Blues",
    annot=False,
    fmt="d"
)

plt.title("Confusion Matrix (Test Set)")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")

plt.show()
```



The confusion matrix shows that most predictions fall along the diagonal, indicating that the model correctly classifies many samples. However, several off-diagonal entries reveal that certain hand signs are frequently confused with visually similar gestures. This suggests that the baseline MLP model struggles to capture subtle spatial patterns in the images. Since MLP treats the image as a flattened vector, it cannot effectively exploit the spatial structure of hand shapes. This limitation motivates the use of convolutional neural networks (CNNs) in the next stage of the project.

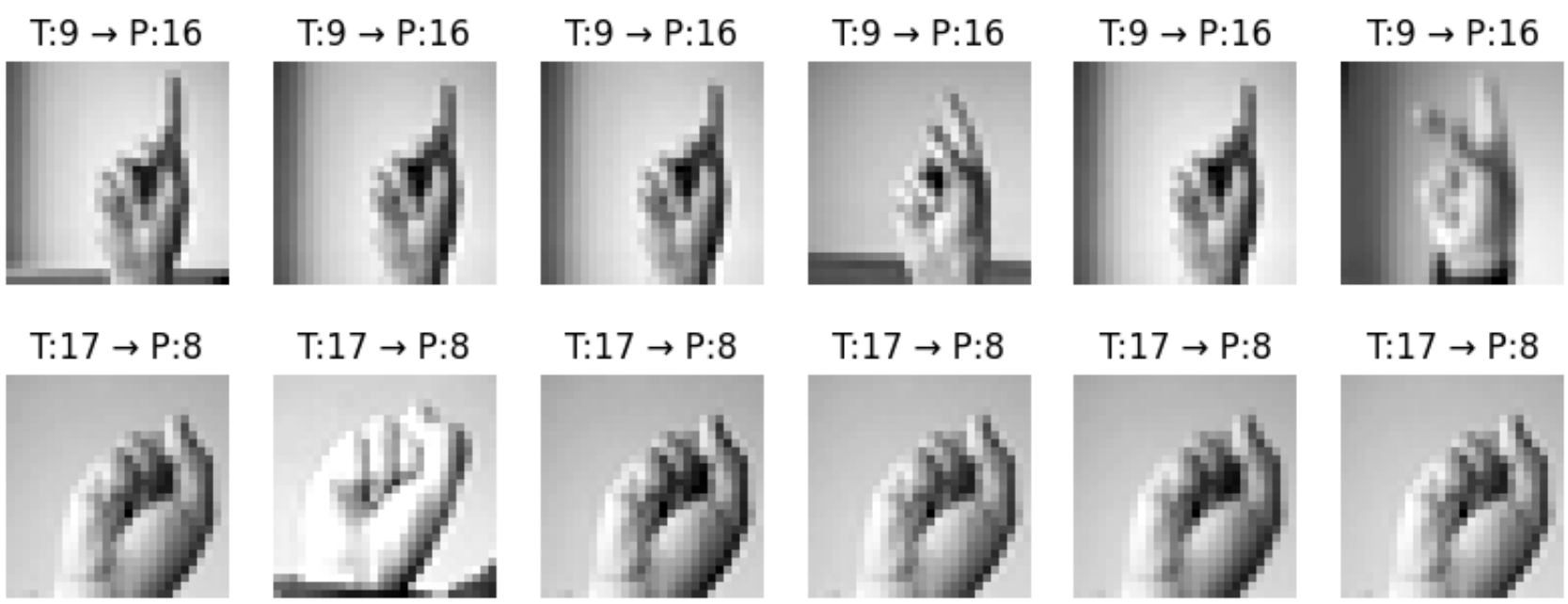
In [ ]:
```
cm_copy = cm.copy()
```

```
np.fill_diagonal(cm_copy, 0)

flat_indices = np.argsort(cm_copy.ravel())[::-1]

top_k = 5
pairs = []

for idx in flat_indices:
    i = idx // cm_copy.shape[1]
    j = idx % cm_copy.shape[1]

    if cm_copy[i, j] > 0:
        pairs.append((i, j, cm_copy[i, j]))

    if len(pairs) == top_k:
        break

print("Most confused class pairs:")
for i, j, v in pairs:
    print(f"True {i} → Predicted {j}: {v} samples")
```

```
Most confused class pairs:
True 23 → Predicted 8: 67 samples
True 9 → Predicted 16: 65 samples
True 18 → Predicted 22: 64 samples
True 11 → Predicted 4: 62 samples
True 20 → Predicted 21: 55 samples
```

In [ ]:

```
import random

def show_misclassified(true_label, pred_label, num_images=6):

    indices = []

    for idx in range(len(all_labels)):
        if all_labels[idx] == true_label and all_preds[idx] == pred_label:
            indices.append(idx)

    if len(indices) == 0:
        print("No samples found")
        return

    selected = random.sample(indices, min(num_images, len(indices)))

    plt.figure(figsize=(10,2))

    for i, idx in enumerate(selected):

        img = X_test.iloc[idx].values.reshape(28,28)

        plt.subplot(1, num_images, i+1)
        plt.imshow(img, cmap="gray")
        plt.title(f"T:{true_label} → P:{pred_label}")
        plt.axis("off")

    plt.show()
show_misclassified(9,16)
show_misclassified(17,8)
```



To better understand the model's errors, we analyzed the most frequently confused class pairs using the confusion matrix. For example, class 9 was often misclassified as class 16, and class 17 was frequently predicted as class 8. These errors suggest that the MLP model struggles to distinguish between visually similar hand gestures. Since the MLP treats each image as a

flattened vector, it cannot effectively capture spatial relationships between pixels. This limitation motivates the use of convolutional neural networks (CNNs), which are better suited for extracting spatial features from images.

6. CNN Model

To improve upon the baseline MLP, we build a convolutional neural network (CNN) for the Sign Language MNIST dataset. Unlike the MLP, the CNN can exploit the spatial structure of 28×28 images through convolution and pooling operations.

In [ ]:
```
import torch.nn.functional as F

# Convert image arrays to PyTorch tensors: (N, H, W, C) -> (N, C, H, W)
X_train_cnn = torch.tensor(X_train_img, dtype=torch.float32).permute(0, 3, 1, 2)
X_val_cnn   = torch.tensor(X_val_img, dtype=torch.float32).permute(0, 3, 1, 2)
X_test_cnn  = torch.tensor(X_test_img, dtype=torch.float32).permute(0, 3, 1, 2)

y_train_cnn = torch.tensor(y_train_mapped.values, dtype=torch.long)
y_val_cnn   = torch.tensor(y_val_mapped.values, dtype=torch.long)
y_test_cnn  = torch.tensor(y_test_mapped.values, dtype=torch.long)

train_dataset_cnn = TensorDataset(X_train_cnn, y_train_cnn)
val_dataset_cnn   = TensorDataset(X_val_cnn, y_val_cnn)
test_dataset_cnn  = TensorDataset(X_test_cnn, y_test_cnn)

train_loader_cnn = DataLoader(train_dataset_cnn, batch_size=128, shuffle=True)
val_loader_cnn   = DataLoader(val_dataset_cnn, batch_size=128, shuffle=False)
test_loader_cnn  = DataLoader(test_dataset_cnn, batch_size=128, shuffle=False)

print(X_train_cnn.shape, X_val_cnn.shape, X_test_cnn.shape)
```

```
torch.Size([23336, 1, 28, 28]) torch.Size([4119, 1, 28, 28]) torch.Size([7172, 1, 28, 28])
```

7. CNN Architecture

Baseline CNN

We first implement a baseline CNN with two convolutional layers, max pooling, and dropout. This design allows the model to learn local spatial features while reducing overfitting.

In [ ]:
```
class CNNBaseline(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),   # 28x28 -> 28x28
            nn.ReLU(),
            nn.MaxPool2d(2),                              # 28x28 -> 14x14

            nn.Conv2d(32, 64, kernel_size=3, padding=1),  # 14x14 -> 14x14
            nn.ReLU(),
            nn.MaxPool2d(2)                               # 14x14 -> 7x7
        )

        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 7 * 7, 128),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(128, 24)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

In [ ]:
```
def train_cnn_model(model, train_loader, val_loader, num_epochs=15, lr=0.001):
    loss_fn = nn.CrossEntropyLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    train_losses = []
    val_losses = []
    train_accuracies = []
    val_accuracies = []

    best_val_acc = 0.0
    best_model_state = None
```

```python
    for epoch in range(num_epochs):
        # training
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for X_batch, y_batch in train_loader:
            optimizer.zero_grad()
            outputs = model(X_batch)
            loss = loss_fn(outputs, y_batch)
            loss.backward()
            optimizer.step()

            running_loss += loss.item() * X_batch.size(0)
            _, predicted = torch.max(outputs, 1)
            total += y_batch.size(0)
            correct += (predicted == y_batch).sum().item()

        train_loss = running_loss / total
        train_acc = correct / total

        # validation
        model.eval()
        val_running_loss = 0.0
        val_correct = 0
        val_total = 0

        with torch.no_grad():
            for X_batch, y_batch in val_loader:
                outputs = model(X_batch)
                loss = loss_fn(outputs, y_batch)

                val_running_loss += loss.item() * X_batch.size(0)
                _, predicted = torch.max(outputs, 1)
                val_total += y_batch.size(0)
                val_correct += (predicted == y_batch).sum().item()

        val_loss = val_running_loss / val_total
        val_acc = val_correct / val_total

        train_losses.append(train_loss)
        val_losses.append(val_loss)
        train_accuracies.append(train_acc)
        val_accuracies.append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_model_state = model.state_dict()

        print(f"Epoch {epoch+1:02d} | "
              f"Train Loss: {train_loss:.4f} | Val Loss: {val_loss:.4f} | "
              f"Train Acc: {train_acc:.4f} | Val Acc: {val_acc:.4f}")

    model.load_state_dict(best_model_state)

    return model, train_losses, val_losses, train_accuracies, val_accuracies, best_val_acc
```

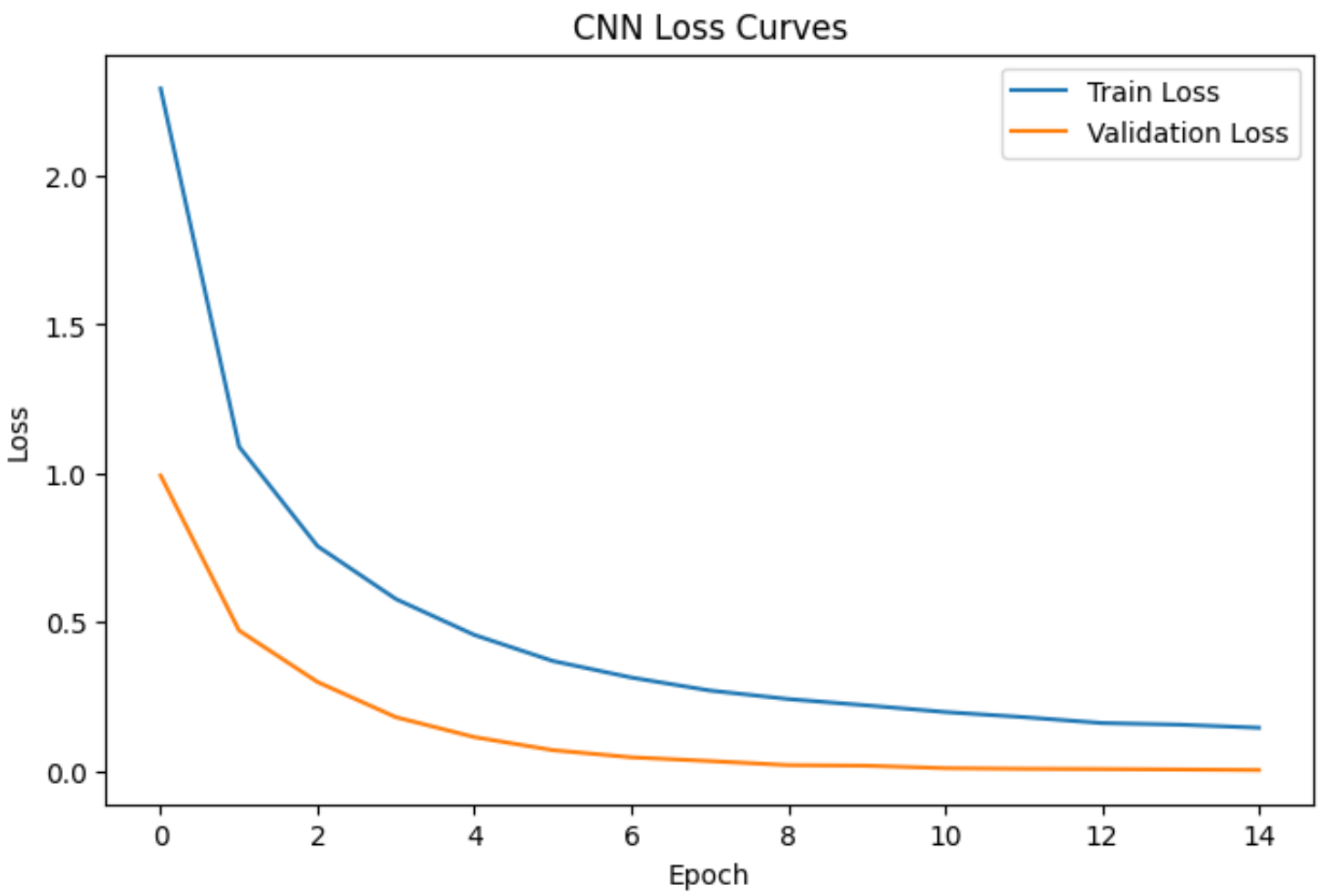
In [ ]:

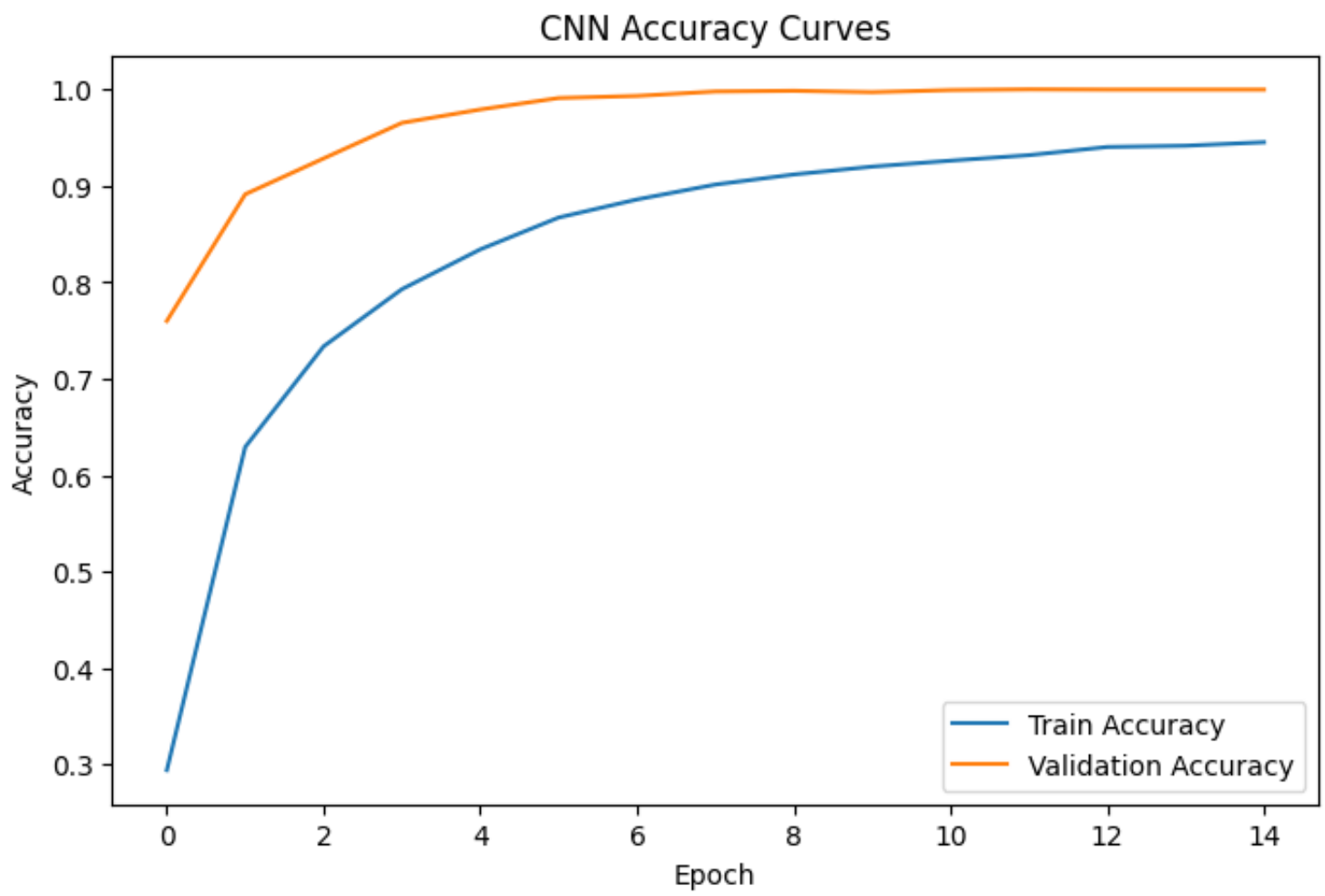
```python
cnn_model = CNNBaseline()

(
    cnn_model,
    cnn_train_losses,
    cnn_val_losses,
    cnn_train_accs,
    cnn_val_accs,
    best_val_acc_cnn
) = train_cnn_model(
    cnn_model,
    train_loader_cnn,
    val_loader_cnn,
    num_epochs=15,
    lr=0.001
)

print("Best CNN Validation Accuracy:", best_val_acc_cnn)
```

```
Epoch 01 | Train Loss: 2.2932 | Val Loss: 0.9930 | Train Acc: 0.2942 | Val Acc: 0.7599
Epoch 02 | Train Loss: 1.0899 | Val Loss: 0.4721 | Train Acc: 0.6292 | Val Acc: 0.8912
Epoch 03 | Train Loss: 0.7556 | Val Loss: 0.2992 | Train Acc: 0.7335 | Val Acc: 0.9284
Epoch 04 | Train Loss: 0.5778 | Val Loss: 0.1808 | Train Acc: 0.7929 | Val Acc: 0.9653
Epoch 05 | Train Loss: 0.4572 | Val Loss: 0.1139 | Train Acc: 0.8341 | Val Acc: 0.9791
Epoch 06 | Train Loss: 0.3697 | Val Loss: 0.0701 | Train Acc: 0.8671 | Val Acc: 0.9910
Epoch 07 | Train Loss: 0.3141 | Val Loss: 0.0461 | Train Acc: 0.8859 | Val Acc: 0.9932
Epoch 08 | Train Loss: 0.2706 | Val Loss: 0.0335 | Train Acc: 0.9013 | Val Acc: 0.9978
Epoch 09 | Train Loss: 0.2418 | Val Loss: 0.0198 | Train Acc: 0.9118 | Val Acc: 0.9985
Epoch 10 | Train Loss: 0.2206 | Val Loss: 0.0179 | Train Acc: 0.9199 | Val Acc: 0.9971
Epoch 11 | Train Loss: 0.1982 | Val Loss: 0.0096 | Train Acc: 0.9261 | Val Acc: 0.9993
Epoch 12 | Train Loss: 0.1811 | Val Loss: 0.0075 | Train Acc: 0.9318 | Val Acc: 1.0000
Epoch 13 | Train Loss: 0.1613 | Val Loss: 0.0065 | Train Acc: 0.9403 | Val Acc: 0.9998
Epoch 14 | Train Loss: 0.1557 | Val Loss: 0.0050 | Train Acc: 0.9415 | Val Acc: 0.9998
Epoch 15 | Train Loss: 0.1454 | Val Loss: 0.0033 | Train Acc: 0.9452 | Val Acc: 0.9998
Best CNN Validation Accuracy: 1.0
```

In [ ]:

```python
# training curve
plt.figure(figsize=(8,5))
plt.plot(cnn_train_losses, label="Train Loss")
plt.plot(cnn_val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("CNN Loss Curves")
plt.legend()
plt.show()

plt.figure(figsize=(8,5))
plt.plot(cnn_train_accs, label="Train Accuracy")
plt.plot(cnn_val_accs, label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("CNN Accuracy Curves")
plt.legend()
plt.show()
```





In [ ]:
```
# test accuracy
cnn_model.eval()
correct_test = 0
total_test = 0

with torch.no_grad():
    for X_batch, y_batch in test_loader_cnn:
        outputs = cnn_model(X_batch)
        _, predicted = torch.max(outputs, 1)
        total_test += y_batch.size(0)
        correct_test += (predicted == y_batch).sum().item()

cnn_test_accuracy = correct_test / total_test
print(f"CNN Test Accuracy: {cnn_test_accuracy:.4f}")
```

```
CNN Test Accuracy: 0.9130
```

Baseline CNN Model Result

The baseline CNN performed well on the Sign Language MNIST dataset. Training loss and validation loss both decreased steadily over 15 epochs, while training accuracy rose to around 95% and validation accuracy reached 100%. The final test accuracy was 91.3%, showing that the CNN can effectively capture spatial features in the hand gesture images. CNN is better suited for image classification compared to MLP. However, since the test accuracy is still lower than the validation accuracy, there is room for improvement in generalization.

Improved CNN

To improve the baseline CNN, we increased the depth of the convolutional feature extractor by adding an additional convolutional layer. This allows the network to learn more complex spatial features from the hand gesture images while keeping the training settings consistent with the baseline model. All other hyperparameters were kept unchanged.

In [ ]:
```
class CNNImproved(nn.Module):
    def __init__(self):
        super().__init__()

        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.ReLU(),

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.ReLU(),

            nn.MaxPool2d(2),   # 28 → 14

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.ReLU(),

            nn.MaxPool2d(2)    # 14 → 7
        )
```

```
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 7 * 7, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, 24)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x
```

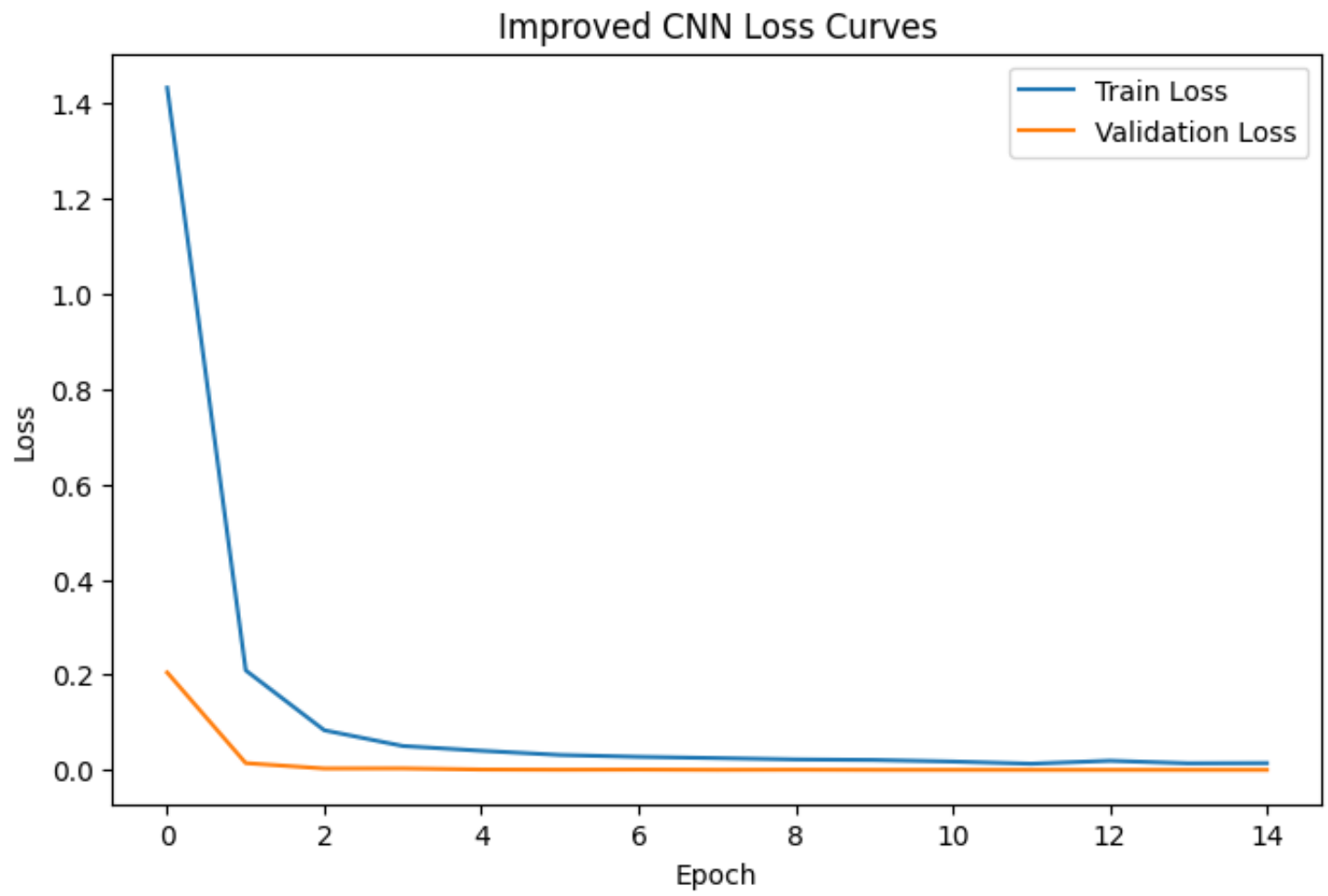
In [ ]:

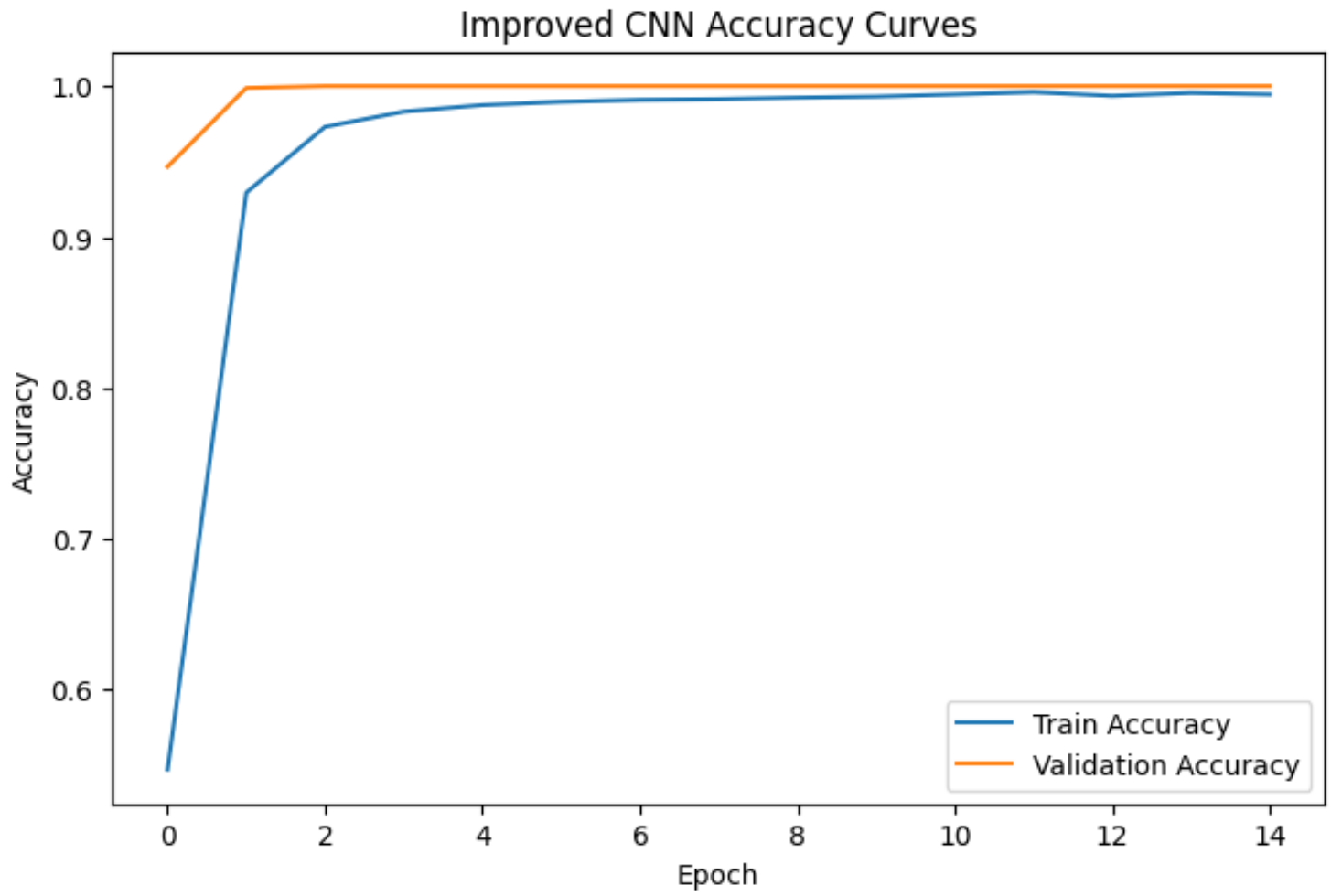
```
cnn_improved = CNNImproved()

(
    cnn_improved,
    imp_train_losses,
    imp_val_losses,
    imp_train_accs,
    imp_val_accs,
    best_val_acc_imp
) = train_cnn_model(
    cnn_improved,
    train_loader_cnn,
    val_loader_cnn,
    num_epochs=15,
    lr=0.001
)

print("Best Improved CNN Validation Accuracy:", best_val_acc_imp)
```

```
Epoch 01 | Train Loss: 1.4331 | Val Loss: 0.2047 | Train Acc: 0.5471 | Val Acc: 0.9466
Epoch 02 | Train Loss: 0.2090 | Val Loss: 0.0143 | Train Acc: 0.9294 | Val Acc: 0.9988
Epoch 03 | Train Loss: 0.0833 | Val Loss: 0.0030 | Train Acc: 0.9730 | Val Acc: 1.0000
Epoch 04 | Train Loss: 0.0502 | Val Loss: 0.0031 | Train Acc: 0.9831 | Val Acc: 1.0000
Epoch 05 | Train Loss: 0.0402 | Val Loss: 0.0008 | Train Acc: 0.9874 | Val Acc: 1.0000
Epoch 06 | Train Loss: 0.0314 | Val Loss: 0.0004 | Train Acc: 0.9895 | Val Acc: 1.0000
Epoch 07 | Train Loss: 0.0275 | Val Loss: 0.0006 | Train Acc: 0.9909 | Val Acc: 1.0000
Epoch 08 | Train Loss: 0.0248 | Val Loss: 0.0001 | Train Acc: 0.9913 | Val Acc: 1.0000
Epoch 09 | Train Loss: 0.0220 | Val Loss: 0.0004 | Train Acc: 0.9923 | Val Acc: 1.0000
Epoch 10 | Train Loss: 0.0209 | Val Loss: 0.0001 | Train Acc: 0.9930 | Val Acc: 1.0000
Epoch 11 | Train Loss: 0.0175 | Val Loss: 0.0001 | Train Acc: 0.9945 | Val Acc: 1.0000
Epoch 12 | Train Loss: 0.0129 | Val Loss: 0.0000 | Train Acc: 0.9960 | Val Acc: 1.0000
Epoch 13 | Train Loss: 0.0192 | Val Loss: 0.0000 | Train Acc: 0.9935 | Val Acc: 1.0000
Epoch 14 | Train Loss: 0.0138 | Val Loss: 0.0000 | Train Acc: 0.9955 | Val Acc: 1.0000
Epoch 15 | Train Loss: 0.0142 | Val Loss: 0.0001 | Train Acc: 0.9945 | Val Acc: 1.0000
Best Improved CNN Validation Accuracy: 1.0
```

In [ ]:

```
# training curve
plt.figure(figsize=(8,5))
plt.plot(imp_train_losses, label="Train Loss")
plt.plot(imp_val_losses, label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Improved CNN Loss Curves")
plt.legend()
plt.show()

plt.figure(figsize=(8,5))
plt.plot(imp_train_accs, label="Train Accuracy")
plt.plot(imp_val_accs, label="Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Improved CNN Accuracy Curves")
plt.legend()
plt.show()
```





In [ ]:
```
# test accuracy
cnn_improved.eval()

correct_test = 0
total_test = 0

with torch.no_grad():
    for X_batch, y_batch in test_loader_cnn:
        outputs = cnn_improved(X_batch)
        _, predicted = torch.max(outputs, 1)
        total_test += y_batch.size(0)
        correct_test += (predicted == y_batch).sum().item()

cnn_improved_test_accuracy = correct_test / total_test

print(f"Improved CNN Test Accuracy: {cnn_improved_test_accuracy:.4f}")
```

```
Improved CNN Test Accuracy: 0.9469
```

Improved CNN Model Result

Although the validation accuracy reached 100% very quickly, which may indicate that the validation set is relatively easy or limited in size. However, the final test accuracy of the improved CNN model is 94.69%, compared with 91.3% for the baseline CNN, suggesting better generalization on the unseen test data. The additional convolutional depth allows the network to learn

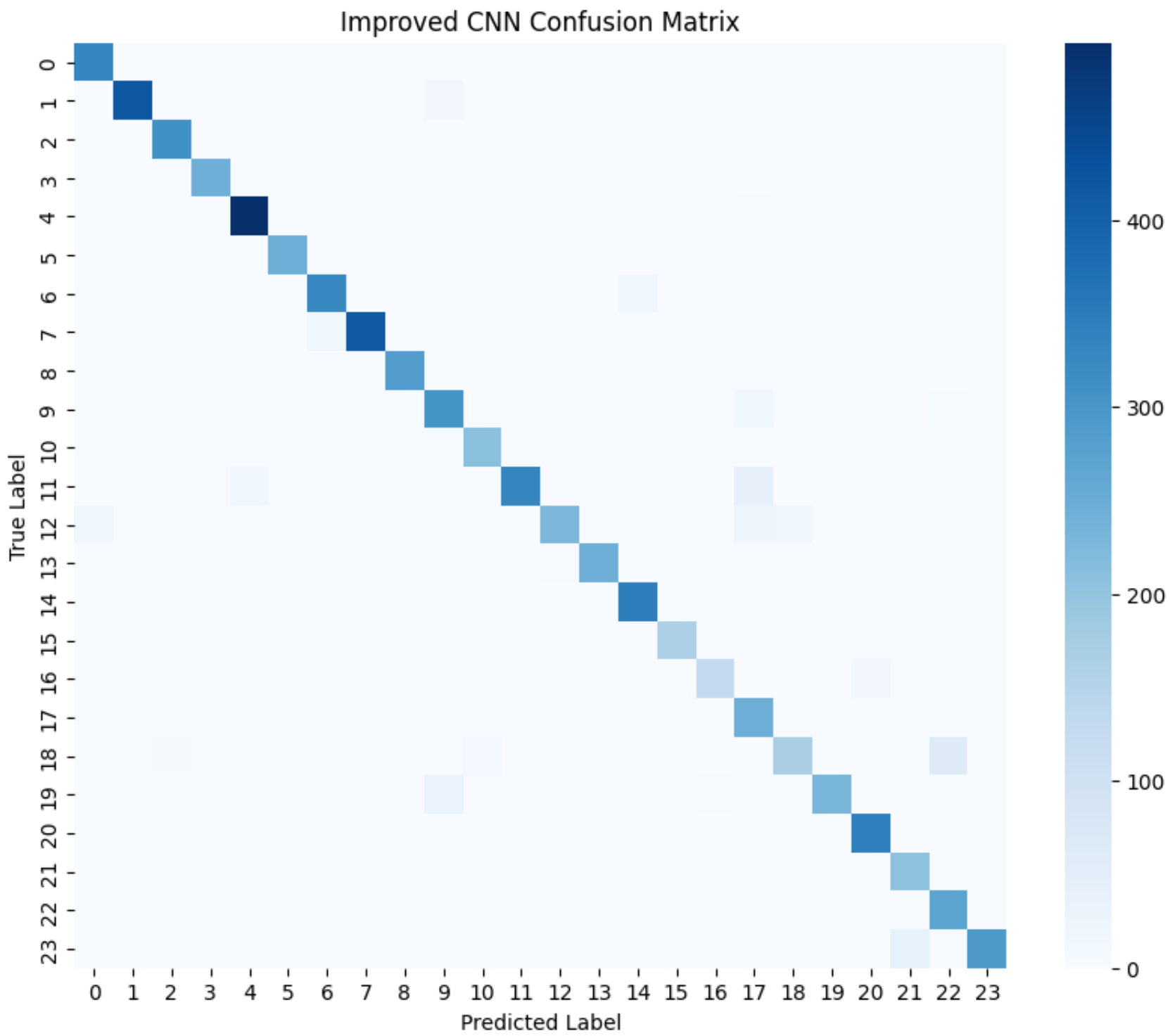
richer spatial features from the hand gesture images, leading to improved classification performance.

Confusion Matrix Analysis

In [ ]:
```
from sklearn.metrics import confusion_matrix
import seaborn as sns
import numpy as np

cnn_improved.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for X_batch, y_batch in test_loader_cnn:
        outputs = cnn_improved(X_batch)
        _, predicted = torch.max(outputs, 1)

        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(y_batch.cpu().numpy())
```

In [ ]:
```
cm = confusion_matrix(all_labels, all_preds)
```

In [ ]:
```
plt.figure(figsize=(10,8))

sns.heatmap(
    cm,
    annot=False,
    cmap="Blues",
    xticklabels=range(24),
    yticklabels=range(24)
)

plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.title("Improved CNN Confusion Matrix")

plt.show()
```



The confusion matrix for the improved CNN shows that most predictions fall along the diagonal, indicating that the model correctly classifies the majority of the gesture images. Compared to the MLP model, the CNN confusion matrix contains noticeably fewer off-diagonal entries, meaning that the number of misclassifications is significantly reduced. This suggests that the CNN is better able to capture spatial patterns in the images, leading to improved classification performance.